ANCHOR GYM SYSTEM

DATABASE NORMALIZATION

UNF → 1NF → 2NF → 3NF

A schema-grounded normalization assessment and final relation inventory

DOCUMENT BASIS	SCOPE
Project database documentation and backend/server.js application schema/workflows	Fourteen current entities; no database modifications

Evidence limitation

No live PostgreSQL catalog export or complete base migration is present in the repository. Legacy primary/unique constraints and some exact column modifiers are therefore unverified. PK* and FK† distinguish logical/application identifiers and references from constraints explicitly declared in backend setup.

Normalization finding

The visible schema already separates accounts, applications, memberships, visits, charges, archives, and support records. No partial or transitive dependency is established strongly enough by the available evidence to justify a speculative decomposition. Repeated historical values are treated as snapshots where the workflow supports that interpretation.

01 | UNNORMALIZED FORM (UNF)

Illustrative source/report layout only — not a table found in the deployed system.

The following wide record embeds repeatable facts inside braces. One account may submit several applications, hold membership history, check in repeatedly, and incur multiple charges. Combining the lists creates non-atomic repeating groups and makes the association between a specific visit and its charge unclear.

Illustrative UNF relation	Repeating group / dependency issue
GYM_REPORT(user_id, full_name, email, phone, applications{application_id, plan, amount, payment_reference, status}, memberships{member_row_id, member_id, plan, start_date, expiration_date}, visits{attendance_id, subject_type, check_in, check_out}, charges{transaction_id, amount, method, transaction_date})	applications{...}, memberships{...}, visits{...}, charges{...} are multi-valued groups in one user row; payment-to-visit association is unclear

Illustrative sample (fictional)

user_id	full_name	applications	visits	charges
42	Alex Sample	A17: Monthly / 1200 / PENDING; A18: Monthly / 1200 / APPROVED	V31: 08:00–09:00; V32: 17:00–18:00	T91: 1200 / GCASH; T92: 50 / CASH

This sample is synthetic and demonstrates repetition only; it is not actual member or transaction data.

Change into 1NF

Move each repeating application, membership, visit, guest, payment, and extension occurrence to its own row-level relation. Preserve the user_id and event identifiers as references. The application already follows this decomposition; no sample UNF table is claimed to exist.

02 | FIRST NORMAL FORM (1NF)

Atomic attributes; one row per entity/event. PK* and FK† mark logical keys/references pending live-catalog verification.

The full 1NF schema is shown below in entity tables. The existing implementation has scalar columns and records repeating events as separate rows. For this assessment, no violation of 1NF is demonstrated by the visible column inventory.

1NF SCHEMAS | ACCOUNT & MEMBERSHIP

Scalar attributes and row-level event records.

USERS

Column	Type	Key
id	INTEGER	PK*
full_name	VARCHAR	–
email	VARCHAR	–
password	VARCHAR	–
phone	VARCHAR	–
role	VARCHAR	–
created_at	TIMESTAMP	–
profile_photo	TEXT	–

APPLICATIONS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
membership_plan	VARCHAR	–
amount	NUMERIC	–
gcash_reference	VARCHAR	–
payment_screenshot	TEXT	–
payment_date	DATE	–
status	VARCHAR	–
rejection_reason	VARCHAR	–
created_at	TIMESTAMP	–

1NF SCHEMAS | ACCOUNT & MEMBERSHIP

Scalar attributes and row-level event records.

MEMBERS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
member_id	VARCHAR	–
registration_type	VARCHAR	–
membership_plan	VARCHAR	–
start_date	DATE	–
expiration_date	DATE	–
status	VARCHAR	–
qr_code	VARCHAR	–
paused_days_remaining	INTEGER	–
archived_at	TIMESTAMP	–
archived_archive_id	BIGINT	FK†

1NF SCHEMAS | ACCOUNT & MEMBERSHIP

Scalar attributes and row-level event records.

MEMBER_TRANSACTIONS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
member_id	INTEGER	FK†
application_id	INTEGER	FK†
amount	NUMERIC	–
payment_method	VARCHAR	–

Column	Type	Key
gcash_reference	VARCHAR	–
transaction_date	TIMESTAMP	–
status	VARCHAR	–
verified_by	INTEGER	–
verification_date	TIME	–
created_at	TIMESTAMP	–

1NF SCHEMAS | ACCOUNT & MEMBERSHIP

Scalar attributes and row-level event records.

ARCHIVED_MEMBERS

Column	Type	Key
archive_id	BIGINT	PK*
original_member_id	INTEGER	FK†
user_id	INTEGER	FK†
member_id	VARCHAR	–
full_name	VARCHAR	–
email	VARCHAR	–
registration_type	VARCHAR	–
membership_plan	VARCHAR	–
start_date	DATE	–
expiration_date	DATE	–
status	VARCHAR	–
qr_code	TEXT	–
archived_at	TIMESTAMP	–

Column	Type	Key
removal_reason	TEXT	–
restored_at	TIMESTAMP	–

1NF SCHEMAS | ATTENDANCE & GUEST BILLING

Scalar attributes and row-level event records.

ATTENDANCE

Column	Type	Key
id	INTEGER	PK*
member_id	INTEGER	FK†
guest_id	INTEGER	FK†
user_type	VARCHAR	–
check_in	TIMESTAMP	–
check_out	TIMESTAMP	–
expires_at	TIMESTAMP	–
expiration_time	TIMESTAMP	–
voided_at	TIMESTAMP	–

GUESTS

Column	Type	Key
id	INTEGER	PK*
full_name	VARCHAR	–
phone	VARCHAR	–
visit_type	VARCHAR	–
amount_paid	NUMERIC	–

Column	Type	Key
payment_method	VARCHAR	–
created_at	TIMESTAMP	–
hours	INTEGER	–
email	VARCHAR	–

1NF SCHEMAS | ATTENDANCE & GUEST BILLING

Scalar attributes and row-level event records.

GUEST_TRANSACTIONS

Column	Type	Key
id	BIGINT	PK
guest_id	BIGINT	FK†
guest_name	TEXT	–
visit_type	TEXT	–
amount	NUMERIC(10,2)	–
payment_method	TEXT	–
transaction_date	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–
transaction_reference	TEXT	–
attendance_id	BIGINT	FK†
extension_type	TEXT	–
additional_hours	INTEGER	–
payment_status	TEXT	–
previous_expiration	TIMESTAMPTZ	–
new_expiration	TIMESTAMPTZ	–

Column	Type	Key
idempotency_key	TEXT	—

1NF SCHEMAS | ATTENDANCE & GUEST BILLING

Scalar attributes and row-level event records.

GUEST_VISIT_EXTENSIONS

Column	Type	Key
id	BIGINT	PK
attendance_id	BIGINT	FK†
guest_id	BIGINT	FK†
extension_type	TEXT	—
additional_hours	INTEGER	—
amount	NUMERIC(10,2)	—
payment_status	TEXT	—
previous_expiration	TIMESTAMPTZ	—
new_expiration	TIMESTAMPTZ	—
transaction_reference	TEXT	—
guest_transaction_id	BIGINT	FK†
idempotency_key	TEXT	—
payment_date	TIMESTAMPTZ	—
created_at	TIMESTAMPTZ	—

1NF SCHEMAS | NOTIFICATIONS & AUTHENTICATION

Scalar attributes and row-level event records.

NOTIFICATIONS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
title	VARCHAR	–
message	VARCHAR	–
is_read	BOOLEAN	–
created_at	TIMESTAMP	–
type	VARCHAR	–

ADMIN_SESSIONS

Column	Type	Key
token_hash	CHAR(64)	PK
user_id	INTEGER	FK
expires_at	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–

1NF SCHEMAS | NOTIFICATIONS & AUTHENTICATION

Scalar attributes and row-level event records.

CUSTOMER_SESSIONS

Column	Type	Key
token_hash	CHAR(64)	PK
user_id	INTEGER	FK
expires_at	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–

MEMBER_QR_TOKENS

Column	Type	Key
token_hash	CHAR(64)	PK
member_id	BIGINT	FK
expires_at	TIMESTAMPTZ	—
consumed_at	TIMESTAMPTZ	—
created_at	TIMESTAMPTZ	—

1NF SCHEMAS | NOTIFICATIONS & AUTHENTICATION

Scalar attributes and row-level event records.

PASSWORD_RESET_TOKENS

Column	Type	Key
token_hash	CHAR(64)	PK
user_id	INTEGER	FK
expires_at	TIMESTAMPTZ	—
used_at	TIMESTAMPTZ	—
attempts	INTEGER	—
verified_at	TIMESTAMPTZ	—
reset_token_hash	CHAR(64)	—
created_at	TIMESTAMPTZ	—

03 | SECOND NORMAL FORM (2NF)

Partial-dependency check against candidate keys.

Rule: a relation must be in 1NF, and each non-key attribute must depend on the whole of every candidate key. A partial dependency requires a composite candidate key and dependence on only part of it.

Evidence-based result

All documented row identifiers are single-column logical keys (id, archive_id, or token_hash). If these are the actual candidate keys and no additional composite candidate key exists, no partial dependency can occur. The project files do not establish the full candidate-key set for the legacy tables, so formal 2NF certification is conditional.

1NF relation	2NF relation structure (PK/FK marked)	2NF result / transformation
USERS	id PK*; full_name; email; password; phone; role; created_at; profile_photo	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
APPLICATIONS	id PK*; user_id FK†; membership_plan; amount; gcash_reference; payment_screenshot; payment_date; status; rejection_reason; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
MEMBERS	id PK*; user_id FK†; member_id; registration_type; membership_plan; start_date; expiration_date; status; qr_code; paused_days_remaining; archived_at; archived_archive_id FK†	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
MEMBER_TRANSACTIONS	id PK*; user_id FK†; member_id FK†; application_id FK†; amount; payment_method; gcash_reference; transaction_date; status; verified_by; verification_date; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
ATTENDANCE	id PK*; member_id FK†; guest_id FK†; user_type; check_in; check_out; expires_at; expiration_time; voided_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
GUESTS	id PK*; full_name; phone; visit_type; amount_paid; payment_method; created_at; hours; email	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
GUEST_TRANSACTIONS	id PK; guest_id FK†; guest_name; visit_type; amount; payment_method; transaction_date; created_at; transaction_reference; attendance_id FK†; extension_type; additional_hours; payment_status; previous_expiration; new_expiration; idempotency_key	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
GUEST_VISIT_EXTENSIONS	id PK; attendance_id FK†; guest_id FK†; extension_type; additional_hours; amount; payment_status; previous_expiration; new_expiration; transaction_reference; guest_transaction_id FK†; idempotency_key; payment_date; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
ARCHIVED_MEMBERS	archive_id PK*; original_member_id FK†; user_id FK†; member_id; full_name; email; registration_type; membership_plan; start_date; expiration_date; status; qr_code; archived_at; removal_reason; restored_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
NOTIFICATIONS	id PK*; user_id FK†; title; message; is_read; created_at; type	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
ADMIN_SESSIONS	token_hash PK; user_id FK; expires_at; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
CUSTOMER_SESSIONS	token_hash PK; user_id FK; expires_at; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.

1NF relation	2NF relation structure (PK/FK marked)	2NF result / transformation
MEMBER_QR_TOKENS	token_hash PK; member_id FK; expires_at; consumed_at; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.
PASSWORD_RESET_TOKENS	token_hash PK; user_id FK; expires_at; used_at; attempts; verified_at; reset_token_hash; created_at	No partial dependency demonstrated; relation/columns unchanged pending catalog key verification.

Functional-dependency example

Relation	Intended dependency	Why no 2NF split is supported
APPLICATIONS	id → user_id, membership_plan, amount, gcash_reference, payment_screenshot, payment_date, status, rejection_reason, created_at	id is a single attribute; there is no proper subset of that key. No composite key/partial dependency is evidenced.
ATTENDANCE	id → member_id, guest_id, user_type, check_in, check_out, expires_at, expiration_time, voided_at	The same single-column-key reasoning applies; the member/guest XOR concern is a referential consistency issue, not a demonstrated 2NF violation.

Change from 1NF to 2NF: none justified by available evidence. No composite-key associative/line-item table is documented; one is not invented merely to demonstrate 2NF.

04 | THIRD NORMAL FORM (3NF)

Transitive-dependency check and final 3NF-oriented schema.

Rule: a relation must be in 2NF, and non-key attributes must not depend transitively on a candidate key through another non-key attribute. No transitive dependency is asserted without evidence of the relevant determinant and business rule.

Area	Dependency considered	Evidence-based treatment
Member transaction / application	If application_id → user_id, amount, reference, then those repeated values may be derivable.	Approval copies application details into a separate payment-ledger row, but the files do not establish whether these are synchronized current values or transaction facts. Keep the implemented relations; confirm the business dependency.
Guest / guest transaction	If guest_id → current guest_name and visit_type, copied fields could be transitive.	Transaction inserts copy labels. Treat as transaction-time snapshots unless the business rule says they track current guest data.
Archived member	Archive values may be derivable from current users/members.	Archive workflow copies a historical snapshot. Retain as point-in-time facts; do not synchronize history.
Extension / guest transaction	Potential overlap in amount, status, references, expiration values.	Rows represent extension event versus charge. No one-to-one or equality dependency is established.
Membership plan / amount	A plan label might determine price if a central plan catalog exists.	No plan catalog or plan → price dependency is evidenced. Keep recorded amounts; do not invent a catalog.
Attendance subject	user_type may depend on which of member_id / guest_id is	This is a polymorphic integrity rule; validate XOR and consistency. It is not

Area	Dependency considered	Evidence-based treatment
	populated.	evidence of a transitive dependency.

Result

The current relations are 3NF-oriented, but formal certification depends on verified candidate keys and functional dependencies. The available evidence does not prove a transitive dependency requiring decomposition. Snapshot fields and repeated amounts may be intentional historical facts rather than normalization defects.

3NF final schema — relation inventory

This final schema matches the entity and attribute inventory in the accompanying ERD. The tables below are the 3NF-oriented implementation as evidenced; no speculative decomposition has been applied.

3NF FINAL SCHEMA | ACCOUNT & MEMBERSHIP

Final relation inventory; same attributes/keys as the ERD.

USERS

Column	Type	Key
id	INTEGER	PK*
full_name	VARCHAR	–
email	VARCHAR	–
password	VARCHAR	–
phone	VARCHAR	–
role	VARCHAR	–
created_at	TIMESTAMP	–
profile_photo	TEXT	–

APPLICATIONS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†

Column	Type	Key
membership_plan	VARCHAR	–
amount	NUMERIC	–
gcash_reference	VARCHAR	–
payment_screenshot	TEXT	–
payment_date	DATE	–
status	VARCHAR	–
rejection_reason	VARCHAR	–
created_at	TIMESTAMP	–

3NF FINAL SCHEMA | ACCOUNT & MEMBERSHIP

Final relation inventory; same attributes/keys as the ERD.

MEMBERS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
member_id	VARCHAR	–
registration_type	VARCHAR	–
membership_plan	VARCHAR	–
start_date	DATE	–
expiration_date	DATE	–
status	VARCHAR	–
qr_code	VARCHAR	–
paused_days_remaining	INTEGER	–
archived_at	TIMESTAMP	–

Column	Type	Key
archived_archive_id	BIGINT	FK†

3NF FINAL SCHEMA | ACCOUNT & MEMBERSHIP

Final relation inventory; same attributes/keys as the ERD.

MEMBER_TRANSACTIONS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
member_id	INTEGER	FK†
application_id	INTEGER	FK†
amount	NUMERIC	–
payment_method	VARCHAR	–
gcash_reference	VARCHAR	–
transaction_date	TIMESTAMP	–
status	VARCHAR	–
verified_by	INTEGER	–
verification_date	TIME	–
created_at	TIMESTAMP	–

3NF FINAL SCHEMA | ACCOUNT & MEMBERSHIP

Final relation inventory; same attributes/keys as the ERD.

ARCHIVED_MEMBERS

Column	Type	Key

Column	Type	Key
archive_id	BIGINT	PK*
original_member_id	INTEGER	FK†
user_id	INTEGER	FK†
member_id	VARCHAR	–
full_name	VARCHAR	–
email	VARCHAR	–
registration_type	VARCHAR	–
membership_plan	VARCHAR	–
start_date	DATE	–
expiration_date	DATE	–
status	VARCHAR	–
qr_code	TEXT	–
archived_at	TIMESTAMP	–
removal_reason	TEXT	–
restored_at	TIMESTAMP	–

3NF FINAL SCHEMA | ATTENDANCE & GUEST BILLING

Final relation inventory; same attributes/keys as the ERD.

ATTENDANCE

Column	Type	Key
id	INTEGER	PK*
member_id	INTEGER	FK†
guest_id	INTEGER	FK†
user_type	VARCHAR	–

Column	Type	Key
check_in	TIMESTAMP	–
check_out	TIMESTAMP	–
expires_at	TIMESTAMP	–
expiration_time	TIMESTAMP	–
voided_at	TIMESTAMP	–

GUESTS

Column	Type	Key
id	INTEGER	PK*
full_name	VARCHAR	–
phone	VARCHAR	–
visit_type	VARCHAR	–
amount_paid	NUMERIC	–
payment_method	VARCHAR	–
created_at	TIMESTAMP	–
hours	INTEGER	–
email	VARCHAR	–

3NF FINAL SCHEMA | ATTENDANCE & GUEST BILLING

Final relation inventory; same attributes/keys as the ERD.

GUEST_TRANSACTIONS

Column	Type	Key
id	BIGINT	PK
guest_id	BIGINT	FK†

Column	Type	Key
guest_name	TEXT	–
visit_type	TEXT	–
amount	NUMERIC(10,2)	–
payment_method	TEXT	–
transaction_date	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–
transaction_reference	TEXT	–
attendance_id	BIGINT	FK†
extension_type	TEXT	–
additional_hours	INTEGER	–
payment_status	TEXT	–
previous_expiration	TIMESTAMPTZ	–
new_expiration	TIMESTAMPTZ	–
idempotency_key	TEXT	–

3NF FINAL SCHEMA | ATTENDANCE & GUEST BILLING

Final relation inventory; same attributes/keys as the ERD.

GUEST_VISIT_EXTENSIONS

Column	Type	Key
id	BIGINT	PK
attendance_id	BIGINT	FK†
guest_id	BIGINT	FK†
extension_type	TEXT	–
additional_hours	INTEGER	–

Column	Type	Key
amount	NUMERIC(10,2)	–
payment_status	TEXT	–
previous_expiration	TIMESTAMPTZ	–
new_expiration	TIMESTAMPTZ	–
transaction_reference	TEXT	–
guest_transaction_id	BIGINT	FK†
idempotency_key	TEXT	–
payment_date	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–

3NF FINAL SCHEMA | NOTIFICATIONS & AUTHENTICATION

Final relation inventory; same attributes/keys as the ERD.

NOTIFICATIONS

Column	Type	Key
id	INTEGER	PK*
user_id	INTEGER	FK†
title	VARCHAR	–
message	VARCHAR	–
is_read	BOOLEAN	–
created_at	TIMESTAMP	–
type	VARCHAR	–

ADMIN_SESSIONS

Column	Type	Key

Column	Type	Key
token_hash	CHAR(64)	PK
user_id	INTEGER	FK
expires_at	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–

3NF FINAL SCHEMA | NOTIFICATIONS & AUTHENTICATION

Final relation inventory; same attributes/keys as the ERD.

CUSTOMER_SESSIONS

Column	Type	Key
token_hash	CHAR(64)	PK
user_id	INTEGER	FK
expires_at	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–

MEMBER_QR_TOKENS

Column	Type	Key
token_hash	CHAR(64)	PK
member_id	BIGINT	FK
expires_at	TIMESTAMPTZ	–
consumed_at	TIMESTAMPTZ	–
created_at	TIMESTAMPTZ	–

3NF FINAL SCHEMA | NOTIFICATIONS & AUTHENTICATION

Final relation inventory; same attributes/keys as the ERD.

PASSWORD_RESET_TOKENS

Column	Type	Key
token_hash	CHAR(64)	PK
user_id	INTEGER	FK
expires_at	TIMESTAMPTZ	–
used_at	TIMESTAMPTZ	–
attempts	INTEGER	–
verified_at	TIMESTAMPTZ	–
reset_token_hash	CHAR(64)	–
created_at	TIMESTAMPTZ	–

VALIDATION NOTES

Consistency, limits, and checks to perform against PostgreSQL.

Check	Current evidence / next validation
ERD consistency	Fourteen final relations and the attributes/keys shown in this document match the accompanying ERD.
Physical keys	Verify primary keys and all candidate/unique keys in the live PostgreSQL catalog; a generated identifier is not proof of a PK.
Foreign keys	Backend setup explicitly declares cascading FKs for admin_sessions.user_id, customer_sessions.user_id, password_reset_tokens.user_id, and member_qr_tokens.member_id. Other links are logical/application references unless catalog inspection proves otherwise.
Attendance XOR	Validate exactly one of member_id and guest_id is populated and that it agrees with user_type.
Application payment uniqueness	Application approval checks duplicate member_transactions rows in code; verify whether application_id has a unique constraint/index.
Snapshots	Confirm the business meaning and retention requirements for archived_members copied facts and guest_transactions guest_name/visit_type.
No database changes	This deliverable documents the existing implementation; it does not alter schema or application behavior.